



Every Kubernetes manifest is the same envelope: **apiVersion** · **kind** · **metadata** · **spec**. The "wish" lives in the spec.

Field	Meaning	Example
apiVersion	Which API group/version serves this kind	v1 (Pod) · apps/v1 (Deployment)
kind	What object you want	Pod, Service, Deployment
metadata	Name · namespace · labels	name: nginx
spec	The desired state	replicas, containers, ports...

The Two-Specs Mystery (Deployment)

```
spec:                                # the DEPLOYMENT's wish
  replicas: 3
  selector: ...
  template:                          # a whole Pod hiding inside!
    metadata: labels: ...
    spec:                             # ← the POD's wish (spec inside spec)
      containers: ...
```

 **Outer spec** = what the Deployment manages: count, selector, template

 **Inner spec** (`spec.template.spec`) = the actual Pod: images, ports, env, resources

 `spec.template` is the **cookie cutter** — the ReplicaSet stamps Pods from it; change anything inside and you trigger a rollout (CH 20)



Indentation is structure in YAML. The most common manifest bug is writing the Pod spec at the wrong nesting level — and the API Server answers with a confusing validation error. Count your two spaces.

[↑ back to top](#)



The STATUS column is a diagnosis. Each state tells you *which stage* failed — before you run a single command.

Status	Meaning	Classic cause	First move
Pending	Accepted, not placed	Not enough CPU/mem · taints · PVC pending	<code>describe pod</code> → events
ContainerCreating	Placed, pulling/wiring	Normal for seconds; stuck = image/CNI issue	<code>describe pod</code> → events
ImagePullBackOff	Can't fetch image	Typo in tag · private registry, no secret	Check image name & pull secret
CrashLoopBackOff	Starts, dies, repeats	App crashes on boot — bad config, missing env	<code>logs --previous</code>
Running	Process is up	— (not necessarily <i>healthy</i> — that's probes)	Test the endpoint
Completed	Finished & exited 0	Job finished — normal for Jobs, odd for servers	Expected? done



The backoff family: `ImagePullBackOff` and `CrashLoopBackOff` are not errors *of* Kubernetes — they're Kubernetes *retrying* after your app/image failed. The real error is one layer deeper, in the events or logs.



Memory hook: Pending = the **Scheduler** said no · ImagePull = the **Registry** said no · CrashLoop = the **App** said no. Three different culprits, three different rooms to search.

[↑ back to top](#)



The STATUS column is a diagnosis. Each state tells you *which stage* failed — before you run a single command.

Status	Meaning	Classic cause	First move
Pending	Accepted, not placed	Not enough CPU/mem · taints · PVC pending	<code>describe pod</code> → events
ContainerCreating	Placed, pulling/wiring	Normal for seconds; stuck = image/CNI issue	<code>describe pod</code> → events
ImagePullBackOff	Can't fetch image	Typo in tag · private registry, no secret	Check image name & pull secret
CrashLoopBackOff	Starts, dies, repeats	App crashes on boot — bad config, missing env	<code>logs --previous</code>
Running	Process is up	— (not necessarily <i>healthy</i> — that's probes)	Test the endpoint
Completed	Finished & exited 0	Job finished — normal for Jobs, odd for servers	Expected? done



The backoff family: `ImagePullBackOff` and `CrashLoopBackOff` are not errors *of* Kubernetes — they're Kubernetes *retrying* after your app/image failed. The real error is one layer deeper, in the events or logs.



Memory hook: Pending = the **Scheduler** said no · ImagePull = the **Registry** said no · CrashLoop = the **App** said no. Three different culprits, three different rooms to search.

[↑ back to top](#)



```
$ kubectl describe pod web-x7k2p      # 1 · full biography...
Events:                               # ...always read the BOTTOM
  Failed to pull image "nginx:1.99"
$ kubectl logs web-x7k2p --previous    # 2 · crashed container's last words
panic: missing REQUIRED_DB_PASSWORD
$ kubectl exec -it web-x7k2p -- sh     # 3 · go inside
$ kubectl get events --sort-by=.lastTimestamp # 4 · cluster timeline
```

DESCRIBE — THE BIOGRAPHY

Config as the cluster sees it, plus the **Events block at the bottom** — the single most information-dense place in debugging.

LOGS --PREVIOUS — THE AUTOPSY

CrashLoopBackOff kills the evidence — `--previous` reads the **last dead container's** output. Forgetting this flag = reading a fresh empty log and panicking.

The 4-step playbook (in order)



- 1 **get pods** — read the STATUS column (CH 23 table)
- 2 **describe pod** — events at the bottom name the failing stage
- 3 **logs --previous** — the app's own error message
- 4 **exec -it -- sh** — poke around inside: DNS, env, files, curl



90% of debugging is reading. The cluster narrates its own failures — through events and logs — if you ask in the right order.

[↑ back to top](#)



```
$ kubectl describe pod web-x7k2p      # 1 · full biography...
Events:                               # ...always read the BOTTOM
  Failed to pull image "nginx:1.99"
$ kubectl logs web-x7k2p --previous    # 2 · crashed container's last words
panic: missing REQUIRED_DB_PASSWORD
$ kubectl exec -it web-x7k2p -- sh     # 3 · go inside
$ kubectl get events --sort-by=.lastTimestamp # 4 · cluster timeline
```

DESCRIBE — THE BIOGRAPHY

Config as the cluster sees it, plus the **Events block at the bottom** — the single most information-dense place in debugging.

LOGS --PREVIOUS — THE AUTOPSY

CrashLoopBackOff kills the evidence — `--previous` reads the **last dead container's** output. Forgetting this flag = reading a fresh empty log and panicking.

The 4-step playbook (in order)



- 1 **get pods** — read the STATUS column (CH 23 table)
- 2 **describe pod** — events at the bottom name the failing stage
- 3 **logs --previous** — the app's own error message
- 4 **exec -it -- sh** — poke around inside: DNS, env, files, curl



90% of debugging is reading. The cluster narrates its own failures — through events and logs — if you ask in the right order.

[↑ back to top](#)

ClusterIPs are invisible from outside the cluster. **NodePort** punches a hole: every Node opens one high port and forwards it to your Service.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: NodePort      # default is ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80        # inside the cluster
      targetPort: 80   # container's port
      nodePort: 30080  # outside — or omit & let k8s pick
```

THE PATH IN



"ANY node" is the superpower — even a Node with **no nginx Pod** opens the door and forwards across.

 Valid range: **30000–32767** (omit `nodePort` and one is auto-assigned)

 Every Node gets the same port — hit `192.168.1.11:30080` or `.13:30080`, same result



NodePort is for demos & edge cases — raw ports, no load balancing across nodes, no TLS story. Production rides **Ingress** (a different book in this series 😊).

[↑ back to top](#)

ClusterIPs are invisible from outside the cluster. **NodePort** punches a hole: every Node opens one high port and forwards it to your Service.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: NodePort      # default is ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80        # inside the cluster
      targetPort: 80   # container's port
      nodePort: 30080  # outside — or omit & let k8s pick
```

THE PATH IN



"ANY node" is the superpower — even a Node with **no nginx Pod** opens the door and forwards across.

 Valid range: **30000–32767** (omit `nodePort` and one is auto-assigned)

 Every Node gets the same port — hit `192.168.1.11:30080` or `.13:30080`, same result



NodePort is for demos & edge cases — raw ports, no load balancing across nodes, no TLS story. Production rides **Ingress** (a different book in this series 😊).

[↑ back to top](#)

Here's the trick nobody tells you: the ClusterIP your Service owns is a **lie**. Nothing is listening on it. No process holds it. It's a **virtual IP (VIP)** — and that's the elegance.

RUSSIAN DOLLS — WHERE THE VIP REALLY GOES

10.96.0.50
the VIP — a fiction

→ DNAT →

10.244.1.5
a real Pod IP

packets "to the VIP" are rewritten **in transit** to a real Pod — the VIP never terminates anything

Address ranges — separate drawers:

Service CIDR **10.96.0.0/12** # VIPs live here (fake, routable only via rules)

Pod CIDR **10.244.0.0/16** # real Pod IPs (per-node slices)

🚫 You can't `ping` a ClusterIP — nothing answers; the rules only rewrite packets *passing through*

👤 Pods died and got new IPs? Nobody notices — everyone still dials the VIP

📅 The ranges **must not overlap** — two drawers, zero confusion



The Service is a stable illusion: an eternal name + eternal IP in front of a set of mortals (Pods) that churn constantly. The illusion is maintained by **kube-proxy** — next chapter.

↑ back to top

Here's the trick nobody tells you: the ClusterIP your Service owns is a **lie**. Nothing is listening on it. No process holds it. It's a **virtual IP (VIP)** — and that's the elegance.

RUSSIAN DOLLS — WHERE THE VIP REALLY GOES

10.96.0.50
the VIP — a fiction

→ DNAT →

10.244.1.5
a real Pod IP

packets "to the VIP" are rewritten **in transit** to a real Pod — the VIP never terminates anything

Address ranges — separate drawers:

Service CIDR **10.96.0.0/12** # VIPs live here (fake, routable only via rules)

Pod CIDR **10.244.0.0/16** # real Pod IPs (per-node slices)

🚫 You can't `ping` a ClusterIP — nothing answers; the rules only rewrite packets *passing through*

👤 Pods died and got new IPs? Nobody notices — everyone still dials the VIP

📅 The ranges **must not overlap** — two drawers, zero confusion



The Service is a stable illusion: an eternal name + eternal IP in front of a set of mortals (Pods) that churn constantly. The illusion is maintained by **kube-proxy** — next chapter.

↑ back to top

kube-proxy does **not** forward a single packet. Its entire job is two verbs: **watch & program**.

✗ WHAT IT DOES NOT DO

- Sit in the traffic path
- Proxy connections through itself
- Touch Pod-to-Pod traffic

✓ WHAT IT DOES

- Watch** the API Server: Services + Endpoints
- Program** iptables/IPVS rules on *its* node
- Re-program the instant Pods change

🔧 The rules it writes (iptables + DNAT)

```
# conceptually, on every node:
if dst == 10.96.0.50:80 # the VIP
  then rewrite dst → 10.244.1.5:80 # pick an endpoint (random)
  else if ... next endpoint # (rule set balances across all)
```



The Illusion, resolved: a Pod dies → Endpoints object updates → every node's kube-proxy rewrites its rules → the **VIP never moved**. Stability on the outside, churn on the inside, and the packets never knew.



One-liner for exams: *kube-proxy watches Services/Endpoints and programs packet-rewriting rules on every node — it never forwards traffic itself.*

↑ back to top

kube-proxy does **not** forward a single packet. Its entire job is two verbs: **watch & program**.

✗ WHAT IT DOES NOT DO

- Sit in the traffic path
- Proxy connections through itself
- Touch Pod-to-Pod traffic

✓ WHAT IT DOES

- Watch** the API Server: Services + Endpoints
- Program** iptables/IPVS rules on *its* node
- Re-program the instant Pods change

🔧 The rules it writes (iptables + DNAT)

```
# conceptually, on every node:
if dst == 10.96.0.50:80 # the VIP
  then rewrite dst → 10.244.1.5:80 # pick an endpoint (random)
  else if ... next endpoint # (rule set balances across all)
```



The Illusion, resolved: a Pod dies → Endpoints object updates → every node's kube-proxy rewrites its rules → the **VIP never moved**. Stability on the outside, churn on the inside, and the packets never knew.



One-liner for exams: *kube-proxy watches Services/Endpoints and programs packet-rewriting rules on every node — it never forwards traffic itself.*

↑ back to top



A Pod runs `curl nginx-svc`. Follow the single packet:

- 1 **DNS** — `nginx-svc` resolves to the VIP `10.96.0.50`
CoreDNS answered from the Service's namespace: `nginx-svc.frontend.svc.cluster.local`
- 2 **Send** — Pod sends to the VIP (nothing is there — remember CH 26)
dest IP: `10.96.0.50` · dest port: `80`
- 3 **DNAT** — the node's iptables rules rewrite the destination
dest becomes a real Pod IP: `10.244.1.5:80`
- 4 **Route** — same node → direct delivery; different node → CNI overlay carries it
the packet leaves kube-proxy's rules and enters the real network
- 5 **Receive** — the Pod sees a normal packet addressed to *its own* IP
it replies — source is the VIP thanks to conntrack, so the client never learns the Pod IP
- 6 **Return** — conntrack rewrites the reply back through the VIP
client believes it talked to `10.96.0.50` the whole time 🤖



The entire machine in one sentence: DNS gives you a stable fake IP; kube-proxy's rules secretly rewrite it to a mortal Pod; conntrack makes the lie symmetric. **That's Service networking — everything else is detail.**

[↑ back to top](#)



A Pod runs `curl nginx-svc`. Follow the single packet:

- 1 **DNS** — `nginx-svc` resolves to the VIP `10.96.0.50`
CoreDNS answered from the Service's namespace: `nginx-svc.frontend.svc.cluster.local`
- 2 **Send** — Pod sends to the VIP (nothing is there — remember CH 26)
dest IP: `10.96.0.50` · dest port: `80`
- 3 **DNAT** — the node's iptables rules rewrite the destination
dest becomes a real Pod IP: `10.244.1.5:80`
- 4 **Route** — same node → direct delivery; different node → CNI overlay carries it
the packet leaves kube-proxy's rules and enters the real network
- 5 **Receive** — the Pod sees a normal packet addressed to *its own* IP
it replies — source is the VIP thanks to conntrack, so the client never learns the Pod IP
- 6 **Return** — conntrack rewrites the reply back through the VIP
client believes it talked to `10.96.0.50` the whole time 🤖



The entire machine in one sentence: DNS gives you a stable fake IP; kube-proxy's rules secretly rewrite it to a mortal Pod; conntrack makes the lie symmetric. **That's Service networking — everything else is detail.**

[↑ back to top](#)



TAB 1 · OBSERVE

```
$ kubectl get deployments -w
$ kubectl get pods -w
$ kubectl get rs -w
```

TAB 2 · CONTROL

```
$ kubectl create deploy web --image=nginx --replicas=2
$ kubectl delete pod <pod-name>
$ kubectl scale deploy web --replicas=4
```

STEP 1 · Create

RUN

```
$ kubectl create deployment web --image=nginx --replicas=2
deployment.apps/web created
```

EXPECTED in TAB 1

```
web 2/2 READY
web-aaaaa 1/1 Running
web-bbbbb 1/1 Running
```

STEP 2 · Kill a Pod

RUN

```
$ kubectl delete pod web-aaaaa
pod "web-aaaaa" deleted
```

EXPECTED

```
web-aaaaa Terminating
web-cccc  ContainerCreating
web-cccc  Running # replacement!
```

STEP 3 · What we proved

A ReplicaSet owns the Pods → deletion is a mismatch, not a tragedy. The **name** died; the **promise** didn't. (Standalone pods: `kubectl run p1 --image=nginx` then delete it — silence. Try it.)

[↑ back to top](#)